Step 4: Flag difficult documents. Image-only PDFs need OCR; scanned Hindi and mixed-language pages also require script-aware review. Charts, equations and complex layouts may need manual transcription or specialised extraction. Mark these files `needs_review` rather than pretending blank or garbled text is good data.

For each successful extraction, save UTF-8 text separately and keep a stable identifier linking it to the original. This link becomes essential when your AI must cite where an answer came from.

Clean without destroying meaning

Raw extraction is not trustworthy by default. Imagine a warranty clause interrupted by a page number, a repeated company footer and a line break that separates "not" from "covered". The model may still produce fluent prose, but its answer could be wrong.

Step 1: Remove boilerplate. Strip recurring headers, footers, cookie notices, website menus and "related articles" sections. Keep meaningful headings, exceptions, table labels and dates. Never delete a sentence merely because it repeats: a repeated safety warning may be intentional.

Step 2: Repair formatting. Join hard-wrapped lines within paragraphs, but preserve paragraph and list boundaries. Normalise whitespace and Unicode consistently. Do not flatten bullet lists into one long sentence; hierarchy often explains whether an exception applies to an entire policy or just one product.

Step 3: Deduplicate cautiously. Calculate a hash for each cleaned document to catch identical copies. For near-duplicates, compare titles, versions and overlapping passages. Keep the authoritative current version, but archive superseded versions when historical questions matter. Two policies that differ by a single "not" are not duplicates.

Step 4: Check language and encoding. Test Indian English, Hindi and Hinglish samples directly. Unicode normalisation is useful, but careless transliteration or replacement can corrupt names, model numbers, rupee symbols and Devanagari text. Preserve the original script wherever possible.

Step 5: Review a sample. Open five randomly chosen processed files beside their originals. Verify that numbers, negations, dates, units, tables and headings survived. Count empty files and suspiciously short extractions. If one extractor mangles a certain layout, fix that class of documents before continuing.

Good cleaning is conservative. Its purpose is to remove accidental noise, not rewrite the author's meaning.

Add metadata and provenance

A paragraph reading "Refunds take seven days" is ambiguous without context. Seven days from what event? Which policy version? Does it apply to appliances, phones or marketplace sellers? The words alone are rarely enough.

Attach metadata to every document: `document_id`, `title`, `source`, `version`, `updated_at`, `language`,



`document_type` and `access_level`. For extracted PDFs, also preserve page numbers. For spreadsheets, retain sheet names and row references. Give each record a stable ID rather than relying on a filename that someone might change.

Store processed documents as Markdown or JSON records. A minimal JSON entry might have a `text` field containing the policy paragraph and a `metadata` object containing the source, version and page. The specific layout matters less than consistency and traceability.

Now run a provenance check. Pick ten random paragraphs. Can you locate each one in the original file? Can you say which version is current? Can you remove one source completely if permission changes? If any answer is no, your data pipeline needs repair before you build search over it.

A useful habit is to maintain a short dataset card, even for a private project: purpose, sources, languages, collection dates, cleaning steps, known gaps and usage permissions. Hugging Face's dataset-card guidance treats licensing, language and limitations as essential context, not decorative metadata.

Understand the dataset formats

Your cleaned corpus is ready for retrieval later, but fine-tuning requires examples of desired behaviour. Treat these as separate outputs.

A raw corpus stores source documents. A question-answer dataset pairs questions with verified answers and references. An instruction dataset adds a task description and a desired response. A conversation dataset contains multiple message turns labelled by role. Preference data compares stronger and weaker responses to the same prompt; it supports other kinds of post-training, not ordinary supervised fine-tuning by itself.